Data Analytics Capstone

Predicting Service Level Agreement Breach in IT Incident Management: An Analysis of Ticket-Level Drivers in an Enterprise ServiceNow Event Log

Final Report

Binesh Balakrishnan

Walsh College

QM640: Data Analytics Capstone

Ms Keya Choudhury Ganguli

Term 3

November 10, 2026

GitHub Repository (Data and Code)

https://github.com/binesh29/QM640_data_analytics_capstone

The repository holds the unchanged source data (data/raw), the incident-level analysis file (data/processed), six Jupyter notebooks that produce every table and figure in this report (notebooks/01 to 06), the Python modules and deployment prototype (src/ and app/), the numeric results as JSON files (results/), and the report figures (figures/). The README explains how to install the pinned library versions and run the notebooks in order; Appendix B lists which notebook produces which result.

Table of Contents

Abstract....4
Introduction....6
Background and Context....6
Problem Statement....7
Purpose of the Study....7
Research Problems / Research Questions....8
Contributions and Expected Value....8
Literature Review....9
Literature Review Approach....9
Summary of Key Literature....9
Materials and Method....12
Data Source and Acquisition....12
Dataset Overview and Inclusion and Exclusion Criteria....14
Data Dictionary....15
Exploratory Data Analysis....16
Research Hypotheses....20
Minimum Sample Size....21
Statistical Methods....22
Database Set-Up and Data Access....23
Data Splitting....23
Model Selection and Justification....24
Training and Test Accuracy and Comparison With Current Methods....24
Architecture Diagram/Workflow....25
System Overview....25
Architecture Diagram....26
Workflow Components....27
Tools and Technologies....29
Results....30
Model Performance....31
Visual Evidence....32
Results by Research Question....36
Overall Interpretation....38
Practical Significance....39
Implementation and User Benefit....39
Deployment Approach....39
System Integration....40
User Interaction....40
Benefits to Users....41
Example Use Case....42
Limitations and Further Improvements....42
Limitations....42
Impact of Limitations....43
Future Improvements....43
Future Scope....44
References....45
Appendix A: Sample Size Code....50
Appendix B: Notebook Map and Run Order....51
Appendix C: Running the Deployment Prototype....51

Abstract

Service level agreement (SLA) breaches in information technology (IT) incident management are counted after the fact, so service desks escalate tickets by priority and intuition rather than evidence. This study asks which ticket-level factors are associated with SLA breach and whether breach risk can be predicted when a ticket is logged. The approach combines statistics with machine learning (ML): χ^2 tests, multivariable logistic regression, and Mantel–Haenszel pooling test priority, reassignment, and knowledge-base use (RQ1 to RQ3), and a priority-only baseline is compared with regularised logistic regression, random forest, and extreme gradient boosting (XGBoost) by the DeLong test (RQ4). The data are a public ServiceNow event log from the University of California, Irvine (UCI) Machine Learning Repository: 141,712 audit events collapsed to N = 24,918 incidents, the unit of analysis, opened between 29 February 2016 and 16 February 2017, of which 36.6% breached. All work used Python with pandas, statsmodels, scikit-learn, and XGBoost on a central processing unit (CPU) in Google Colab and on a laptop; no graphics processing unit (GPU) was needed. All four null hypotheses were rejected at $\alpha = .05$. Critical and High incidents breached 98.1% and 99.5% of the time against 15.9% for Low ones (Cramér's $V = 0.228$). Each reassignment multiplied the adjusted odds of breach by 1.95. The raw knowledge-base effect (odds ratio 6.24) shrank to 1.17 once a March 2016 process change was accounted for. On a time-ordered test set of 7,476 incidents, a random forest using 17 attributes known at logging reached an area under the receiver operating characteristic curve (AUC) of 0.785, against 0.552 for the priority rule ($z = 38.45$, $p < .001$); flagging the 20% highest-risk tickets caught 46.6% of breaches at 2.3 times the base rate. A working prototype stores the data in SQLite, serves recalibrated risk scores through a representational state transfer (REST) application programming interface (API), and ranks the queue in a dashboard, so a

ServiceNow business rule can score each new ticket and duty managers can act before the SLA clock runs out.

Introduction

Every enterprise information technology (IT) service desk promises its clients resolution times in a service level agreement (SLA) and reports each month how often it missed them. Those misses damage the client relationship, can trigger financial penalties, and absorb management time, yet the report arrives after the damage is done. This study treats SLA breach as something that can be explained and predicted from the ticket itself, using real ServiceNow data.

Background and Context

Incident management is the IT service management (ITSM) process that restores normal service after an unplanned interruption. On a platform such as ServiceNow, each incident is logged, given a priority derived from its impact and urgency, routed to a support group, worked, resolved, and closed, and every change is written to an audit trail. The IT Infrastructure Library (ITIL) describes this lifecycle, and organisations that adopt it report better service quality but continuing difficulty in measuring what drives process performance (Marrone & Kolbe, 2011).

In managed services, the process is governed by an SLA that fixes target resolution times by priority. The stakeholders are the end users who lose service, the analysts and resolver groups who handle tickets, the service delivery manager accountable for SLA attainment, and the client who pays. A breach is therefore both an operational and a commercial event.

Two behaviours are widely blamed for delay: reassignment, when a ticket passes between groups before it reaches one that can fix it, which routing research treats as waste (Shao et al., 2008), and whether recorded knowledge, such as a knowledge-base article, is used. Artificial intelligence for IT operations (AIOps) has also raised interest in anticipating service problems from operational data (Dang et al., 2019), and predictive process monitoring forecasts the

outcome of running cases from event logs (Teinemaa et al., 2019; van der Aalst, 2016). What is missing is a test of the factors managers act on against the contractual outcome, and a prediction made early enough to change it.

Problem Statement

Service desks escalate incidents by priority, but they have no evidence on which ticket-level factors drive SLA breach, and they learn about at-risk tickets only when the SLA clock is nearly exhausted. The objective of this study is to explain and predict Y = SLA breach (binary: 1 = breached, 0 = met) for each incident using X = priority, reassignment count, knowledge-base use, and 17 attributes recorded when the ticket is logged, controlling for service category, over the 24,918 incidents opened between 29 February 2016 and 16 February 2017 in one enterprise ServiceNow instance. Success is evaluated by rejection of each null hypothesis at $\alpha = .05$ with an effect size (Cramér's V, odds ratio) for RQ1 to RQ3, and for RQ4 by a test-set area under the receiver operating characteristic (ROC) curve (AUC) significantly above that of a priority-only model (DeLong test), supported by precision and recall at a fixed review capacity and by the Brier score.

Purpose of the Study

The study has two aims. The explanatory aim is to estimate how strongly priority, reassignment, and knowledge-base use are associated with SLA breach after allowing for priority and service category (RQ1 to RQ3). The predictive aim is to classify each new incident by its breach risk at logging, using only information available then, with machine learning (ML): regularised logistic regression, random forest, and extreme gradient boosting (XGBoost) are compared with the priority rule by AUC (RQ4). The best model is then built into a working scoring service and dashboard.

Research Problems / Research Questions

All four questions share the same dependent variable, SLA breach.

Research Question 1 (RQ1): Does the SLA breach rate differ significantly across the four priority levels (Critical, High, Moderate, Low)?

Research Question 2 (RQ2): Is the number of reassignments associated with the likelihood of SLA breach after controlling for priority and service category?

Research Question 3 (RQ3): Is knowledge-base use associated with the likelihood of SLA breach after controlling for priority and service category?

Research Question 4 (RQ4): Can an ML classifier that uses only attributes available when a ticket is logged predict SLA breach with a significantly higher AUC than a priority-only baseline?

Contributions and Expected Value

Practical contribution. Four managerial beliefs become tested, quantified statements, and each new ticket receives a risk score with a rule for how many tickets to review daily. **Technical contribution.** SLA breach is framed as classification at logging rather than during execution, with a leakage-free feature set, time-ordered evaluation, and a demonstration of how a process change distorts tests and validation if ignored. **Value to stakeholders.** Managers gain evidence for routing and escalation policy, resolvers gain earlier warning, and clients gain fewer breaches. Because every ServiceNow instance records these fields, the method can be repeated elsewhere.

Literature Review

Literature Review Approach

Sources were found by keyword searches of Google Scholar, Crossref, arXiv, and the ACM, IEEE, and Springer libraries, using terms such as “incident management”, “ticket routing”, “SLA violation prediction”, “predictive process monitoring”, “deep learning tabular data”, “large language model incident”, and “data leakage”. Peer-reviewed papers, two widely cited preprints, and standard methods texts were included when they addressed IT service operations, prediction from event logs, deep-learning (DL) or large language model (LLM) approaches to operations, or the methods used here. Details were checked against Crossref or arXiv. Table 1 links 15 key sources to the research questions and design decisions.

Summary of Key Literature

Table 1

Literature Relevance Matrix

Author (Year)	Domain / context	Dataset / setting	Method(s)	Key findings	How it supports the RQs / project decisions
Marrone & Kolbe (2011)	ITSM frameworks	Survey of ITIL-adopting organisations	Empirical survey	ITIL brings benefits, but measuring process performance stays hard	Motivates measuring SLA drivers (all RQs)
Shao et al. (2008)	Ticket routing	Problem-ticket transfer histories	Resolution-sequence mining (Markov model)	Mined routing cuts transfers needed to reach a resolver	Treats reassignment as a cost driver (RQ2)
Botezatu et al. (2015)	Ticket dispatching	Enterprise incident tickets	Multi-view clustering	Clustering ticket views improves first-time dispatch	First assignment group as a predictor (RQ2, RQ4)
Leitner et al. (2010)	Service computing	Composite web services	Runtime monitoring with ML prediction	SLA violations can be predicted and prevented during execution	SLA breach as a prediction target (RQ4)
Amaral et al. (2018a, 2019)	Process mining	This incident log	Filter and wrapper attribute selection	Selected attributes improve completion-time prediction	Same data; breach classification is the gap (RQ4)
Teinemaa et al. (2019)	Predictive process monitoring	24 tasks from 9 event logs	Benchmark of 11 outcome-prediction methods	No method dominates; tree ensembles strong	Outcome-prediction design; model shortlist (RQ4)

Author (Year)	Domain / context	Dataset / setting	Method(s)	Key findings	How it supports the RQs / project decisions
Tax et al. (2017)	Predictive monitoring (DL)	Public event logs	Long short-term memory (LSTM) networks	LSTMs predict next events and remaining time of running cases	DL needs event sequences, absent at logging (RQ4)
Evermann et al. (2017)	Process prediction (DL)	Public challenge event logs	Recurrent neural networks	DL learns process behaviour without hand-made features	Considered and rejected for at-logging data (RQ4)
Bukhsh et al. (2021)	Predictive monitoring (DL)	Public event logs	Transformer network	Attention models match or beat LSTMs on sequence tasks	Future scope: in-flight risk updates
Grinsztajn et al. (2022); Shwartz-Ziv & Armon (2022)	ML on tabular data	Benchmark tabular datasets	Tree ensembles versus DL	Tree ensembles match or beat DL on typical tabular data	Choice of random forest and XGBoost over DL (RQ4)
Ahmed et al. (2023)	Cloud incidents (LLM)	40,000+ Microsoft incidents	Fine-tuned generative LLMs	LLMs can draft root-cause and mitigation text for engineers	LLM assistant layer in future scope
Chen et al. (2024)	Cloud incidents (agentic LLM)	Microsoft production incidents	RCACopilot: tool-collecting agent plus LLM	Automated data collection with LLM reasoning speeds root-cause analysis	Agent that acts on high-risk tickets (future scope)
Yao et al. (2023)	LLM agents	Question answering and decision tasks	ReAct: interleaved reasoning and tool actions	Reasoning-plus-acting agents outperform reasoning-only prompts	Pattern for a triage agent calling the risk-scoring service
Kaufman et al. (2012)	Data-mining practice	Competitions and real projects	Formal definition of leakage	Leakage inflates measured accuracy	Excludes counts that accrue after logging (RQ4)
DeLong et al. (1988); Saito & Rehmsmeier (2015)	Classifier evaluation	Clinical and imbalanced data	Correlated-AUC test; precision–recall analysis	Paired AUC test; precision–recall curves more informative under imbalance	H_{04} test; precision–recall reporting (RQ4)

Note. Key findings are paraphrased from each source. DL = deep learning; LLM = large language model; ITIL = IT Infrastructure Library.

Routing and reassignment in ITSM. Ticket-handling research treats every transfer as lost time. Shao et al. (2008) mine transfer sequences to recommend the next group, and Botezatu et al. (2015) cluster tickets so that they reach the right group first time. Both measure success by transfers or dispatch accuracy rather than by the SLA outcome, and both rely on text or sequence data. With surveys showing that ITIL adopters struggle to measure performance drivers (Marrone & Kolbe, 2011), this work motivates RQ2, but none estimates how much each reassignment changes the odds of breach with priority held constant.

Predicting outcomes from event logs. Predictive process monitoring forecasts how a running case will end (van der Aalst, 2016). Teinemaa et al. (2019) benchmark 11 outcome-prediction methods on 24 tasks and find no single winner, with tree-ensemble classifiers among the strongest, and Leitner et al. (2010) predict SLA violations in web services from runtime data. The log used here has been studied only for completion-time prediction (Amaral et al., 2018a, 2019). In-flight predictions use events such as reassignments that have already happened, so they arrive after the routing decision that matters most; this study therefore predicts at logging.

Deep learning for process data. Tax et al. (2017) and Evermann et al. (2017) use recurrent networks to predict the next activity and remaining time of running cases, and Bukhsh et al. (2021) show that a transformer, based on attention rather than recurrence, can match or beat them. These models need a sequence of events, and at logging there is only one record per incident. For single-row tabular data with many categorical fields, two benchmarks find that tree ensembles match or outperform DL with far less tuning and compute (Grinsztajn et al., 2022; Shwartz-Ziv & Armon, 2022). This is why RQ4 uses random forest and XGBoost rather than a neural network, and why a central processing unit (CPU) without a graphics processing unit (GPU) suffices.

Large language models and agents in operations. Ahmed et al. (2023) fine-tune generative models on more than 40,000 cloud incidents to draft root-cause and mitigation suggestions, and Chen et al. (2024) combine automated diagnostic data collection with LLM reasoning in RCACopilot, an agentic system that gathers evidence before proposing a cause. Such agents build on the ReAct pattern, in which a model alternates reasoning with tool calls (Yao et al., 2023). These systems work on free text after an incident is understood; they do not estimate the probability of an SLA breach. The approaches are complementary: a calibrated risk

score can decide which tickets deserve an agent's or engineer's attention, which is how an LLM layer appears in this study's future scope. The source log has no free-text fields, so LLM methods could not be applied to it directly.

Validity and evaluation. Leakage, using information that would not exist at prediction time, is a common cause of overstated accuracy (Kaufman et al., 2012); here, reassignment, reopen, and update counts accumulate after logging and are excluded from prediction. For imbalanced outcomes, AUC measures ranking quality (Fawcett, 2006), precision–recall measures describe the minority class better (Saito & Rehmsmeier, 2015), and the Brier score checks probabilities (Brier, 1950); two AUCs on one test set are compared with the DeLong et al. (1988) test. For time-ordered data, evaluation that respects the order, such as forward chaining, is recommended (Bergmeir & Benítez, 2012), and distorted classifier probabilities can be corrected by Platt scaling (Niculescu-Mizil & Caruana, 2005). For the explanatory questions, logistic regression (Hosmer et al., 2013) and Mantel–Haenszel pooling (Mantel & Haenszel, 1959) adjust for confounders, and the Holm (1979) procedure controls pairwise error.

Gap addressed. The literature optimises routing, predicts process outcomes during execution, and applies LLMs to diagnosis, but no study found tests priority, reassignment, and knowledge use against SLA breach in an ITSM log, or predicts breach at logging against the priority rule practitioners already use. This study fills that gap and adds a deployable scoring service.

Materials and Method

Data Source and Acquisition

The data is the Incident Management Process Enriched Event Log donated by Amaral et al. (2018b) to the University of California, Irvine (UCI) Machine Learning Repository

(https://archive.ics.uci.edu/dataset/498/incident+management+process+enriched+event+log; https://doi.org/10.24432/C57S4H). It is an audit log extracted from the ServiceNow instance of a real IT company, enriched with fields from the platform database and anonymised by the donors. It was downloaded as a single comma-separated values (CSV) file; no web scraping was needed. The Creative Commons Attribution 4.0 (CC BY 4.0) licence permits redistribution with attribution, so the unchanged file is stored in the project repository (Figure 1 and Figure 2).

Figure 1

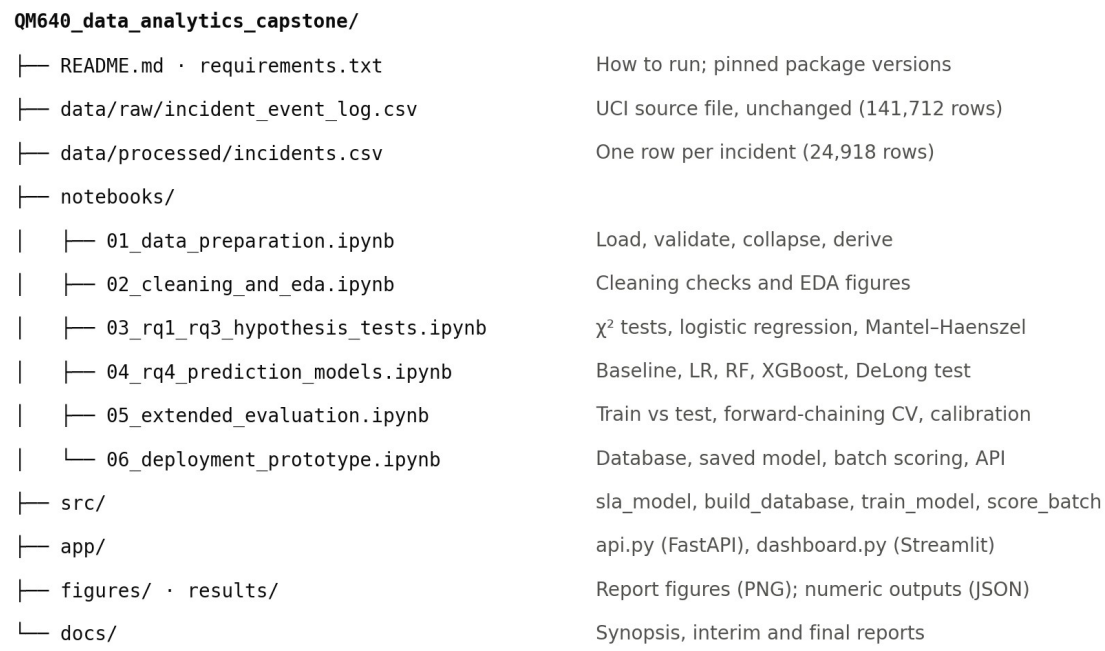
Extract of the Raw Event Log (First Six Events, Eight of 36 Columns)

number	incident_state	reassignment_count	made_sla	opened_at	priority	knowledge	resolved_at
INC0000045	New	0	true	29/2/2016 01:16	3 - Moderate	true	29/2/2016 11:29
INC0000045	Resolved	0	true	29/2/2016 01:16	3 - Moderate	true	29/2/2016 11:29
INC0000045	Resolved	0	true	29/2/2016 01:16	3 - Moderate	true	29/2/2016 11:29
INC0000045	Closed	0	true	29/2/2016 01:16	3 - Moderate	true	29/2/2016 11:29
INC0000047	New	0	true	29/2/2016 04:40	3 - Moderate	true	1/3/2016 09:52
INC0000047	Active	1	true	29/2/2016 04:40	3 - Moderate	true	1/3/2016 09:52

Note. Each incident (number) appears on several rows as it changes state. From Incident Management Process Enriched Event Log [Data set], by C. Amaral, M. Fantinato, and S. Peres, 2018, UCI Machine Learning Repository (https://doi.org/10.24432/C57S4H). CC BY 4.0.

Figure 2

Structure of the GitHub Repository



Note. Folder structure of https://github.com/binesh29/QM640_data_analytics_capstone. The SQLite database and the saved model are rebuilt by notebook 06 and are not stored in the repository.

Dataset Overview and Inclusion and Exclusion Criteria

The file has 141,712 rows and 36 columns; each row is a snapshot of an incident after an update, so an incident appears on 5.7 rows on average. Analysing rows would count incidents several times, so the log was collapsed to one row per incident, the unit of analysis: 24,918 incidents opened between 29 February 2016 and 16 February 2017 (98.1% between March and May 2016). Each incident keeps its first record, which holds what was known at logging (RQ4), and its final record, which holds the outcome and accumulated counts (RQ1 to RQ3). The dependent variable is sla_breach = 1 when made_sla is false on the final record; 9,115 incidents (36.6%) breached. Median resolution time is 1.1 hours when made_sla is true and 201.7 hours when it is false, which confirms that true means the SLA was met.

Inclusion criteria. Every incident was included, because each has a recorded outcome and the data are 6.3 times the minimum sample size; five events with an undocumented state code (−100) were kept because they do not change the outcome. **Exclusion criteria.** Five columns that are 98% to 100% empty (caused_by, vendor, cmdb_ci, rfc, problem_id) were excluded, apart from a flag for a missing configuration item. Analyst identifiers were excluded for privacy, so the study rates tickets, not people. Fields known only after logging (reassignment, reopen, and update counts, resolution and closure times, closure code) were excluded from the RQ4 model to prevent leakage. No incidents were removed as outliers.

Data Dictionary

Table 2 defines the variables used, with their type, range, handling of missing values, and role. All 36 source columns are listed in the repository README.

Table 2

Data Dictionary (Incident Level)

Variable	Definition	Type	Values / range	Missing values	Role
number	Incident identifier	String	24,918 unique	None	Key
sla_breach (derived)	1 if made_sla is false on the final record	Binary	0 / 1; mean 0.366	None	Y, all RQs
priority	Calculated from impact and urgency	Ordinal	1 Critical – 4 Low	None	X (RQ1); control
reassignment_count	Times the group or analyst changed	Integer	0–27; median 0	None	X (RQ2); excluded from RQ4
knowledge	Knowledge-base article used	Boolean	true / false	None	X (RQ3); RQ4 sensitivity
category	First-level affected service	Nominal	52 levels (final); 57 at logging; 31 groups	0.2% at logging; “Missing” level	Control (RQ2–RQ3); RQ4
subcategory	Second-level service	Nominal	246 labels	0.3%; “Missing” level	RQ4
u_symptom	User-reported symptom	Nominal	500 labels	23.0%; “Missing” level	RQ4
assignment_group	First support group	Nominal	69 labels	3.6%; “Missing” level	RQ4

Variable	Definition	Type	Values / range	Missing values	Role
impact, urgency	Impact and urgency of the incident	Ordinal	1 High – 3 Low	None	RQ4
location, contact_type, notify	Affected location; channel; notification	Nominal	219; 5; 2 levels	0.2% (location); none	RQ4
u_priority_confirmation	Priority double-checked	Boolean	true / false	None	RQ4
cmdb_ci_missing (derived)	No configuration item recorded	Binary	0 / 1	None	RQ4
caller_freq (derived)	Training-period incidents of the caller, log(1 + n)	Numeric	≥ 0	Unseen caller = 0	RQ4
hour_sin, hour_cos, weekend, day of week (derived)	Cyclical hour, weekend flag, and weekday of opened_at	Numeric / binary	−1 to 1; 0 / 1; 0–6	None	RQ4
opened_at	Opening time	Datetime	29 Feb 2016 – 16 Feb 2017	None	Split order; features
resolution_hours (derived)	resolved_at − opened_at	Continuous	0 – 8,070 h	1,556 left missing	Descriptive only

Note. Label counts and missing shares are at logging (first record) unless marked final; variables with the log_ prefix in the code are first-record values. Y = dependent variable; X = independent variable. Adapted from Amaral et al. (2018b).

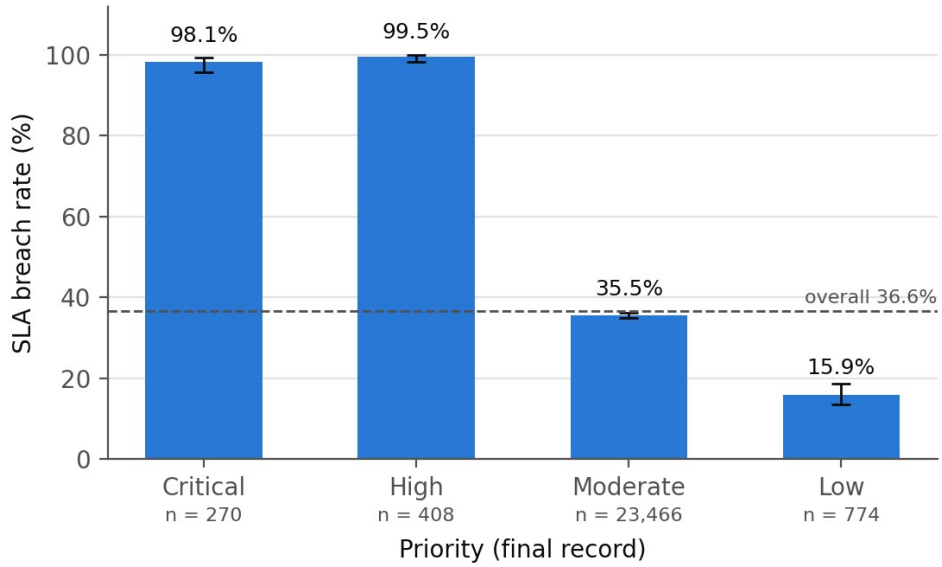
Exploratory Data Analysis

Exploratory data analysis (EDA) began with the distributions. The numeric variables are strongly right-skewed: the median incident is never reassigned, is updated four times, and is resolved in 22.1 hours, while mean resolution time is 178.2 hours because a minority stay open for weeks. Medians, rates, and odds ratios are therefore reported rather than means, and no method used assumes normality. Four patterns shaped the analysis.

Priority. Critical and High incidents breach almost always (98.1% and 99.5%), Moderate incidents 35.5%, and Low incidents 15.9% (Figure 3). Moderate incidents are 94.2% of the volume and account for 91.3% of all breaches, so most of the business impact, and most of the room for a better rule, lies inside a single priority level.

Figure 3

SLA Breach Rate by Priority

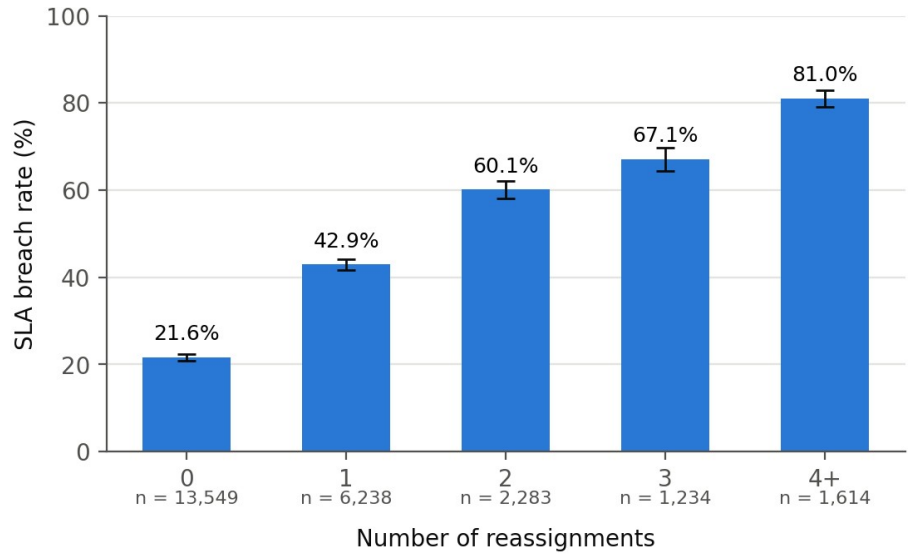


Note. Error bars are Wilson 95% confidence intervals; the dashed line is the overall rate (36.6%). *n* = 270, 408, 23,466, and 774.

Reassignment. The breach rate rises with every hand-off, from 21.6% for incidents never reassigned to 42.9% after one, 60.1% after two, and 81.0% after four or more (Figure 4). Reassigned incidents are 45.6% of the total but 67.9% of breaches.

Figure 4

SLA Breach Rate by Number of Reassignments

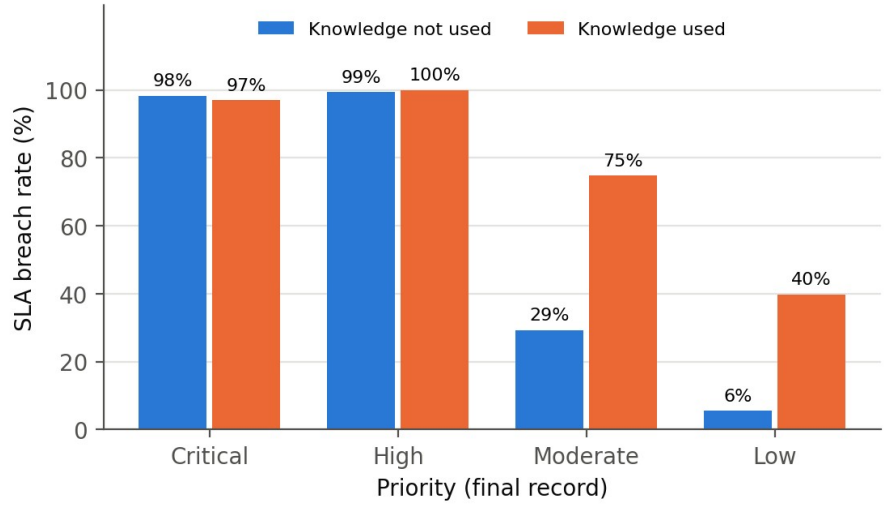


Note. Error bars are Wilson 95% confidence intervals.

Knowledge-base use. Within Moderate and Low priority, incidents that used a knowledge article breached far more often (75% against 29%, and 40% against 6%; Figure 5). Because it is implausible that reading an article causes a breach, the timing of knowledge use was examined.

Figure 5

SLA Breach Rate by Knowledge-Base Use Within Each Priority

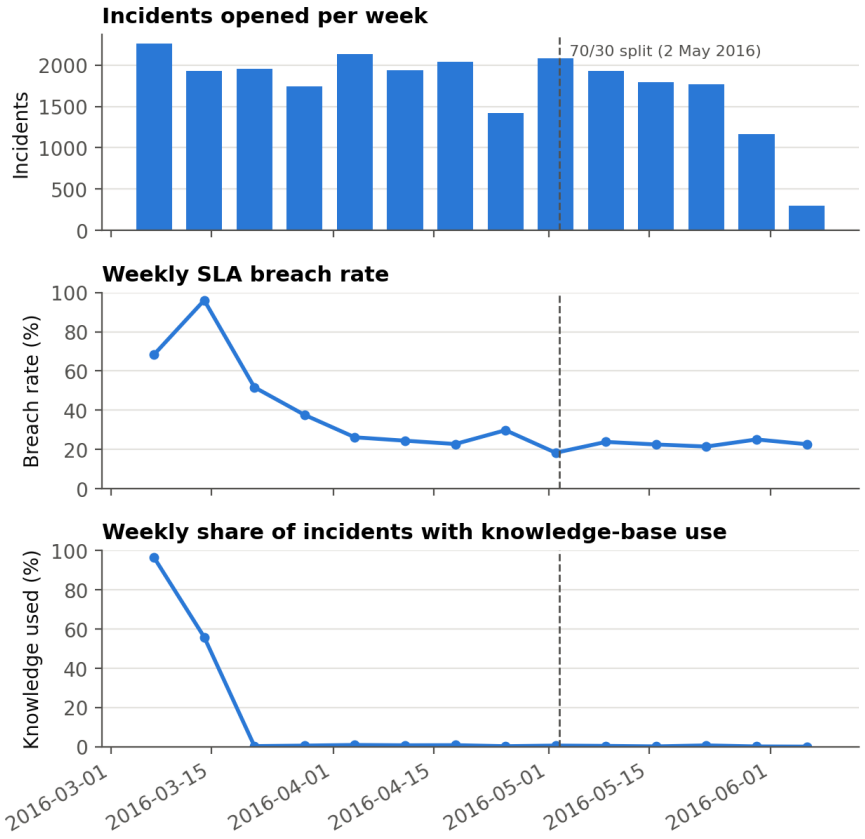


Note. Bars show the share of incidents that breached, with and without knowledge-base use. Data from Amaral et al. (2018b).

A structural break in March 2016. Weekly volume is stable, but the breach rate exceeds 68% in early March and falls to 18% to 30% from April, while the knowledge flag drops from 97% of incidents to under 1% (Figure 6). Before 21 March, 58.6% of 5,906 incidents carry the flag and 73.1% breached; afterwards, 0.5% carry it and 25.2% breached. The likely cause is a change in how knowledge use was recorded, plus an early backlog. The flag therefore marks the period, which confounds RQ3, and the breach rate differs between the RQ4 training (42.3%) and test (23.3%) periods; both are handled below.

Figure 6

Weekly Incident Volume, SLA Breach Rate, and Knowledge-Base Use



Note. Weeks end on the date shown; weeks with fewer than 300 incidents are omitted. The dashed line marks the 70/30 temporal split used for RQ4.

Research Hypotheses

Each research question is tested at the 5% significance level ($\alpha = .05$). H_0 denotes the null and H_a the alternative hypothesis; the subscript gives the research question; β_1 is the logistic regression coefficient of the variable of interest, and e^β_1 is its odds ratio (OR) (Table 3).

Table 3

Null and Alternative Hypotheses by Research Question

RQ	Null hypothesis (H_0)	Alternative hypothesis (H_a)	Statistical method
RQ1	H_{01}: SLA breach is independent of priority	H_{a1}: SLA breach is associated with priority	χ^2 test of independence (4 × 2); Holm-adjusted pairwise tests
RQ2	H_{02}: OR for reassignment count = 1 ($\beta_1 = 0$), given priority and category	H_{a2}: OR ≠ 1 ($\beta_1 \neq 0$)	χ^2 test; multivariable logistic regression (Wald z)
RQ3	H_{03}: OR for knowledge use = 1 ($\beta_1 = 0$), given priority and category	H_{a3}: OR ≠ 1 ($\beta_1 \neq 0$)	χ^2 test; logistic regression; Mantel–Haenszel
RQ4	H_{04}: AUC of best model = AUC of priority-only baseline	H_{a4}: AUC of best model > AUC of baseline	DeLong test for correlated AUCs (one-sided)

Minimum Sample Size

Minimum sample sizes were computed in advance with $\alpha = .05$, power = 0.80, and Cohen's (2013) small effect size $w = 0.10$. For a χ^2 test, $N = \lambda / w^2$, where λ is the noncentrality parameter at which the χ^2 distribution with the test's degrees of freedom reaches 80% power (Faul et al., 2007). For RQ2 and RQ3, N is divided by $(1 - R^2)$, where R^2 is the variance in the variable of interest explained by the controls. For RQ4, the test set must estimate an AUC of A = 0.75 within ±0.03; the standard error (SE) follows Hanley and McNeil (1982), $SE^2 = [A(1 - A) + (n_1 - 1)(Q_1 - A^2) + (n_0 - 1)(Q_2 - A^2)] / (n_1 n_0)$, with $Q_1 = A / (2 - A)$ and $Q_2 = 2A^2 / (1 + A)$, and n grows until $1.96 \times SE \leq 0.03$. Dividing by the 30% test share gives the total (Table 4; code in Appendix A).

Table 4

Minimum Sample Size by Research Question

RQ	Method	Parameters	Computation	Minimum N
RQ1	χ^2 power, 4 × 2	w = 0.10; df = 3; λ = 10.903	10.903 / 0.10^2 = 1,090.3	1,091
RQ2	χ^2 power, 2 × 2, adjusted	w = 0.10; df = 1; λ = 7.849; R^2 = 0.047	785 / (1 − 0.047) = 823.7	824
RQ3	χ^2 power, 2 × 2, adjusted	w = 0.10; df = 1; R^2 = 0.154	785 / (1 − 0.154) = 927.9	928
RQ4	AUC precision (Hanley & McNeil)	A = 0.75; ±0.03; prevalence 0.366; 30% test	test *n* = 1,191; 1,191 / 0.30	3,970
All	*N* = max(RQ1–RQ4)	Available: 24,918 incidents	max(1,091; 824; 928; 3,970)	3,970

Note. α = .05 and power = 0.80 throughout. R^2 values were observed in the data. Results are rounded up.

RQ4 is binding, so the minimum is *N* = 3,970. The 24,918 incidents collected are 6.3 times this, so all were used and every question is adequately powered. The volume also suits the analysis as run: the smallest expected cell in the RQ1 table is 98.8, far above 5, and although the test set's breach rate (23.3%) is lower than assumed, its 7,476 incidents are 4.4 times the 1,681 needed at that rate, giving a 95% confidence half-width of about ±0.012 for the best AUC.

Statistical Methods

With a binary outcome and categorical predictors, RQ1 uses the χ^2 test of independence with Cramér's V as effect size, because with 24,918 incidents almost any difference is significant; pairwise tests use Holm (1979) correction. Reassignment and knowledge use vary with priority and category, so RQ2 and RQ3 use multivariable logistic regression, logit P(Y = 1) = $\beta_0 + \beta_1 X + \Sigma\gamma_k$(priority, category), with adjusted odds ratio e^β_1 and a Wald z test (Hosmer et al., 2013); categories under 30 incidents are pooled, leaving 31 groups. Mantel–Haenszel pooling and the Breslow–Day test check robustness across strata (Mantel & Haenszel, 1959). Further checks add the opening week, band reassignment with a reassignment × priority interaction (likelihood-ratio test), and estimate the knowledge effect before and after the March break. RQ4 uses the one-sided DeLong et al. (1988) test of correlated AUCs.

Database Set-Up and Data Access

The analysis notebooks read the two CSV files with pandas. For deployment, src/build_database.py loads both into a SQLite database, data/db/itsm.sqlite, so that scoring and the dashboard work against a relational store as they would against a ServiceNow reporting replica (Table 5). The script stores "?" as NULL, indexes opening time and assignment group, and defines a view, v_open_queue, that joins each incident to its latest risk score. The model, batch scorer, application programming interface (API), and dashboard access data through structured query language (SQL), for example SELECT * FROM incidents WHERE opened_at >= ? AND opened_at < ? for each scoring window. The database rebuilds in about four seconds and, at 58 MB, is not stored in GitHub.

Table 5

SQLite Database Tables and Views

Object	Rows	Content	Written / read by
event_log (table)	141,712	Raw UCI event log, 36 columns, "?" stored as NULL	build_database.py / audit queries
incidents (table)	24,918	One row per incident: first-record and final-record fields, sla_breach	build_database.py / train_model.py, score_batch.py
risk_scores (table)	7,428	Incident number, time scored, model version, probability, band, action	score_batch.py / dashboard, audit queries
v_open_queue (view)	7,428	Incidents joined to risk scores, ordered by risk	Dashboard (app/dashboard.py)

Note. Row counts for risk_scores and v_open_queue reflect the replay of incidents opened in May 2016 (notebook 06).

Data Splitting

A random split would let the model learn from tickets logged after those it is tested on, and would mix the two regimes around the March break. Incidents were therefore sorted by opening time and split 70/30: the earliest 17,442 (opened before 11:30 on 2 May 2016, breach rate 42.3%) train the models and the latest 7,476 (23.3%, mostly May 2016) test them once.

Hyperparameters were tuned by five-fold stratified cross-validation (CV) on the training set, and a five-fold forward-chaining CV, in which each fold trains on all earlier incidents and validates on the next 2,907, respects the time order (Bergmeir & Benítez, 2012). For the deployed model, the first 80% of the training period (13,953 incidents) fit the random forest and the latest 20% (3,489, 18 April to 2 May 2016) fit the probability recalibration. Caller frequency uses fitting data only, so no test information leaks into the features.

Model Selection and Justification

Four models were compared. **Model 1, priority-only logistic regression (LR),** reproduces the rule service desks already use, so beating it is the meaningful benchmark. **Model 2, regularised LR,** is interpretable through odds ratios, and its L1 or L2 penalty controls overfitting across 255 encoded columns. **Model 3, random forest** (Breiman, 2001), averages many decision trees and captures interactions, such as a group that is risky only for some categories, without specifying them. **Model 4, XGBoost** (Chen & Guestrin, 2016), adds trees sequentially to correct earlier errors and is often the most accurate on tabular data. Tree ensembles were preferred to DL because the at-logging data are one row of mostly categorical fields, where trees match or beat neural networks (Grinsztajn et al., 2022; Shwartz-Ziv & Armon, 2022), and sequence models such as LSTMs (Tax et al., 2017) need event histories that do not yet exist. Balanced class weights handle imbalance. The selection rule, fixed in advance, was the highest test AUC, with stability between validation and test as tiebreaker.

Training and Test Accuracy and Comparison With Current Methods

Table 6 compares training and test performance. The current method, escalation by priority, reaches a test AUC of 0.552; its accuracy of 0.781 is the highest only because it flags almost nothing, catching 7.0% of breaches at a 0.5 threshold. Regularised LR is almost identical

on training and test data (0.783 against 0.779), so it does not overfit. Both tree ensembles fit the training data far more closely (AUC about 0.92) than the test data; the random forest still generalises best (0.785), while XGBoost loses most (0.738). Flexible models trained on a period with a higher breach rate are expected to behave this way, which is why selection rests on the test set.

Table 6

Training Versus Test Performance of the Four Models

Model	Train AUC	Test AUC	Train accuracy	Test accuracy	Train F1	Test F1	Test Brier
Model 1: Priority-only LR (current rule)	0.533	0.552	0.594	0.781	0.085	0.130	0.240
Model 2: Regularised LR	0.783	0.779	0.705	0.722	0.661	0.523	0.185
Model 3: Random forest	0.924	0.785	0.846	0.729	0.817	0.532	0.181
Model 4: XGBoost	0.922	0.738	0.841	0.696	0.814	0.481	0.195

Note. Training set n = 17,442 (breach rate 0.423); test set n = 7,476 (0.233). Accuracy and F1 use a 0.5 threshold. F1 = harmonic mean of precision and recall; Brier = mean squared error of predicted probabilities (lower is better).

Architecture Diagram/Workflow

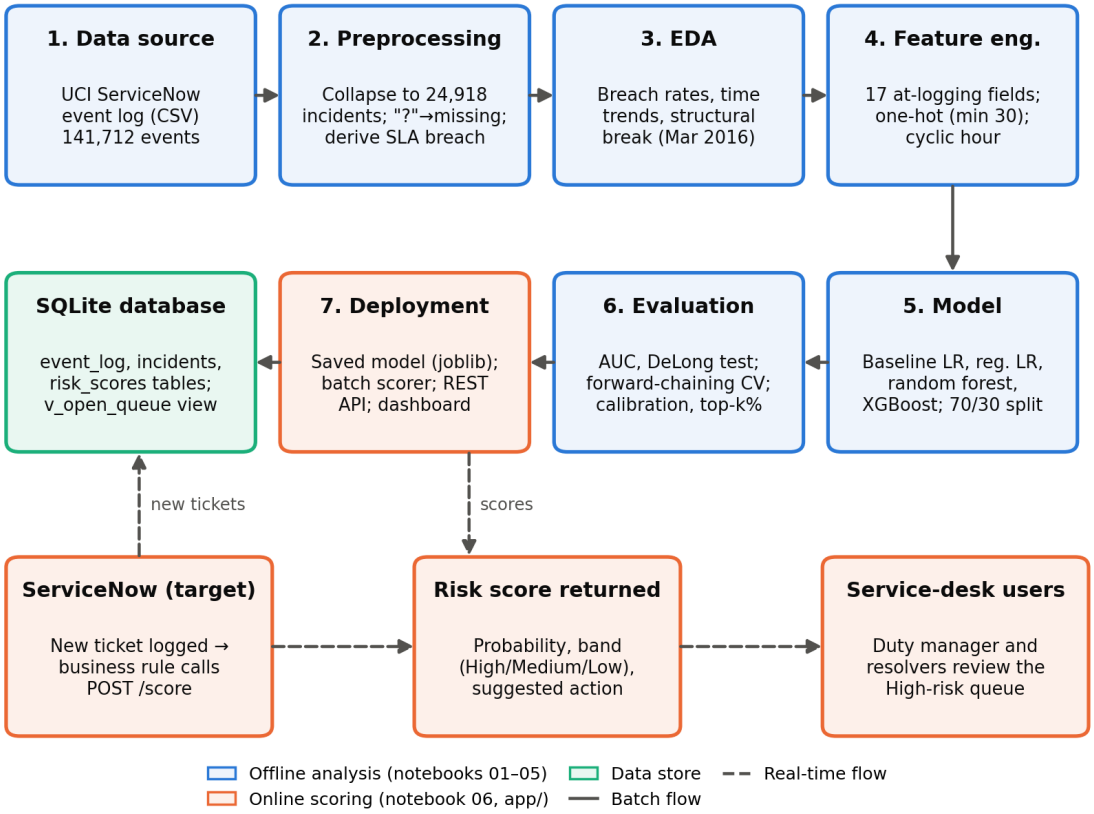
System Overview

The system turns a raw ServiceNow audit log into a risk score for each new ticket. An offline pipeline (notebooks 01 to 05) prepares and explores the data, engineers features, and trains and evaluates the models. An online path (notebook 06 and app/) saves the chosen model, scores tickets from the database in batches or one at a time through a representational state transfer (REST) API, and presents a ranked queue to users (Figure 7).

Architecture Diagram

Figure 7

End-to-End Workflow of the Proposed System



Note. Blue boxes are offline analysis stages, orange boxes are online scoring components, and the green box is the data store. Solid arrows show batch flow; dashed arrows show the real-time flow when ServiceNow calls the scoring API.

As shown in Figure 7, data moves left to right through preprocessing, EDA, and feature engineering, then right to left through modelling, evaluation, and deployment, which writes scores back to the database. In production, ServiceNow would send each new ticket to the scoring API and display the returned risk to users.

Workflow Components

Data Ingestion

The source is one CSV file of 141,712 event records with 36 columns, mixing identifiers, categorical attributes, Boolean flags, counts, and timestamps. Notebook 01 reads it with pandas, checks the expected shape and the count of 24,918 distinct incidents, and parses timestamps as day-first dates. For deployment, the same file and the processed incident file are loaded into SQLite (Table 5).

Data Preprocessing

Events are sorted by incident, update counter, and update time; five tied pairs are ordered by update time, so first and final records are unique and no incident is duplicated. **Missing values.** The "?" code becomes missing. Missing categorical values at logging, such as symptom (23.0%) and assignment group (3.6%), form an explicit "Missing" level rather than being imputed, because an unrouted ticket is informative; resolution time, missing for 1,556 incidents, is left missing because it is only descriptive. **Outliers.** The interquartile range (IQR) rule flags 2,848 incidents with more than two reassignments (maximum 27) and 812 resolved after more than 1,000 hours. These genuine difficult cases are the subject of the study, so they were retained, with reassignment also analysed in bands (0, 1, 2, 3+) and resolution time on a log scale. **Encoding.** Categorical features are one-hot encoded, with levels seen fewer than 30 times in training pooled into an "infrequent" level, giving 255 columns. **Scaling.** Numeric features are standardised for the LR models; tree models use them unscaled because scaling does not change their splits.

Exploratory Data Analysis

Notebook 02 produces the summary statistics and Figures 3 to 6. EDA established the polarity of the outcome, the concentration of breaches in Moderate priority, the dose–response pattern of reassignment, and the March 2016 break, which led directly to the period checks for RQ3 and the time-ordered validation for RQ4.

Feature Engineering

RQ4 uses 17 features, all from the first event record so that they are known at logging (Table 7). Six engineered features fall into four groups: the caller's incident count in the training period, log-transformed, which replaces 5,244 caller identifiers; hour of day as sine and cosine terms, so that 23:00 and 00:00 are close; a weekend flag with day of week; and a flag for a missing configuration item. Knowledge use is excluded from the main model because it marks the period, and is tested in a sensitivity model.

Table 7

Feature Set for the Predictive Model (RQ4)

Feature	Original / engineered	Encoding	Reason for inclusion
Priority	Original	One-hot	Current rule; only feature of Model 1
Impact, urgency	Original	One-hot	Finer detail than priority
Category, subcategory, symptom	Original	One-hot, rare pooled, Missing level	Service area and reported problem
Assignment group (first)	Original	One-hot, rare pooled, Missing level	Routing quality at logging
Location, contact type, notify, priority confirmed	Original	One-hot / binary	Context of the request
Caller frequency	Engineered	log(1 + n), training data only	Frequent callers differ; avoids 5,244 dummies
Hour (sine, cosine), weekend, day of week	Engineered	Numeric / binary / one-hot	Daytime queues breach more
Configuration item missing	Engineered	Binary	Data-quality signal

Note. Excluded to prevent leakage: reassignment, reopen, and update counts; resolution and closure times; closure code; final-record priority.

Model Development

The models were built as scikit-learn pipelines combining encoding and estimation, so identical transformations apply to training, test, and new data. Grid search with five-fold stratified CV on AUC tuned the LR penalty and strength C; the random forest's minimum leaf size and features per split (400 trees); and XGBoost's depth and learning rate (400 trees, 80% row and column sampling). The selected settings were C = 1 with L1, a minimum leaf of 2 with √p features, and depth 7 with learning rate 0.1. All runs use random seed 42.

Model Evaluation

The primary metric is test-set AUC, the quantity in H_{04}, compared by the DeLong test. Precision–recall AUC (PR-AUC), precision, recall, and F1 describe the breach class, and the Brier score measures probability accuracy. Because a service desk can review only a fixed share of tickets, the models are also compared at capacity thresholds: precision and recall when the 10%, 20%, or 30% highest-risk tickets are flagged. Calibration was checked by comparing predicted and observed breach rates by decile.

Deployment

Notebook 06 rebuilds the database, trains the deployable model (random forest plus Platt recalibration), saves it as a single file with joblib, scores incidents in batches into the risk_scores table, and exercises the API. The API (FastAPI) exposes POST /score and GET /health; the dashboard (Streamlit) reads the scored queue. Commands are given in Appendix C, and the design is described under Implementation and User Benefit.

Tools and Technologies

Table 8 lists the software and computing environment. All analysis ran on a CPU; no GPU was used or needed, because tree ensembles on 24,918 rows train in seconds.

Table 8

Tools, Libraries, and Computing Environment

Layer	Tool (version)	Use
Language	Python 3.11 or later	All analysis and deployment code
Data handling	pandas 3.0.2, NumPy 2.4.4	Loading, collapsing, feature engineering
Statistics	SciPy 1.17.1, statsmodels 0.15.0	χ^2 tests, logistic regression, Mantel–Haenszel
ML	scikit-learn 1.8.0, XGBoost 3.2.0	Pipelines, LR, random forest, CV, metrics; boosting
Visualisation	Matplotlib 3.10.9	All figures
Database	SQLite (Python sqlite3)	event_log, incidents, risk_scores
Deployment	joblib, FastAPI, Uvicorn, Streamlit	Saved model, REST API, dashboard
Compute	Google Colab standard CPU runtime; Windows laptop (CPU only)	Notebook execution; no GPU
Version control	Git and GitHub	Code, data, results, reports

Note. Exact versions are pinned in requirements.txt. Each notebook runs in under ten minutes on a CPU.

Results

All results come from notebooks 03 to 06, and every number can be regenerated from the repository. Intervals are 95% confidence intervals (CI), and p-values below .001 are reported as $p < .001$.

Model Performance

Table 9

Model Performance Comparison (Temporal Test Set, n *= 7,476)*

Model	Features used	Validation method	CV AUC	Test AUC [95% CI]	PR-AUC	Precision / recall at top 20%	Key observation
Model 1: Priority-only LR	Priority only	Temporal 70/30	0.533	0.552 [0.545, 0.558]	0.289	0.302 / 0.259	Current rule; barely above chance
Model 2: Regularised LR	17 at-logging	Temporal 70/30; 5-fold CV	0.768	0.779 [0.768, 0.791]	0.532	0.536 / 0.460	Close second; most stable over time
Model 3: Random forest (best)	17 at-logging	Temporal 70/30; 5-fold CV	0.790	**0.785 [0.773, 0.796]**	0.536	0.542 / 0.466	Highest test AUC and PR-AUC
Model 4: XGBoost	17 at-logging	Temporal 70/30; 5-fold CV	0.787	0.738 [0.725, 0.751]	0.481	0.477 / 0.410	Largest drop from CV to test

Note. CV AUC = mean five-fold stratified cross-validated AUC in the training set. CI = DeLong 95% confidence interval. Top 20% = the 1,495 test incidents with the highest predicted risk; chance precision is 0.233. PR-AUC chance level is 0.233.

The random forest is the best model: it has the highest test AUC (0.785) and PR-AUC (0.536), and its AUC barely moves between CV and test (Table 9). Against the priority rule its advantage is 0.233 AUC points (DeLong $z = 38.45$, one-sided $p < .001$). Regularised LR is a close second; the difference from the random forest is small and not significant (0.005, $z = 1.66$, two-sided $p = .097$), whereas both beat XGBoost significantly (random forest by 0.047, $z = 11.76$, $p < .001$).

Forward-chaining CV confirms the ranking and shows why random-fold CV was optimistic (Table 10). The fold that validates on 19 to 30 March, the weeks of the break, is the weakest for every model, and XGBoost is the weakest ML model in every fold. Averaged over the five time-ordered folds, regularised LR (0.722) and random forest (0.713) are close, and both are far above the priority rule (0.548); in the most recent fold, closest to deployment conditions, both reach 0.788.

Table 10

Forward-Chaining Cross-Validation AUC Within the Training Period

Fold (validation period, breach rate)	Priority LR	Regularised LR	Random forest	XGBoost
1 (9–19 Mar 2016, 77.9%)	0.561	0.704	0.705	0.660
2 (19–30 Mar 2016, 35.1%)	0.527	0.600	0.556	0.545
3 (30 Mar–8 Apr 2016, 25.5%)	0.531	0.751	0.744	0.682
4 (8–20 Apr 2016, 23.2%)	0.538	0.768	0.772	0.687
5 (20 Apr–2 May 2016, 22.0%)	0.582	0.788	0.788	0.759
Mean (SD)	0.548 (0.023)	0.722 (0.075)	0.713 (0.093)	0.667 (0.077)

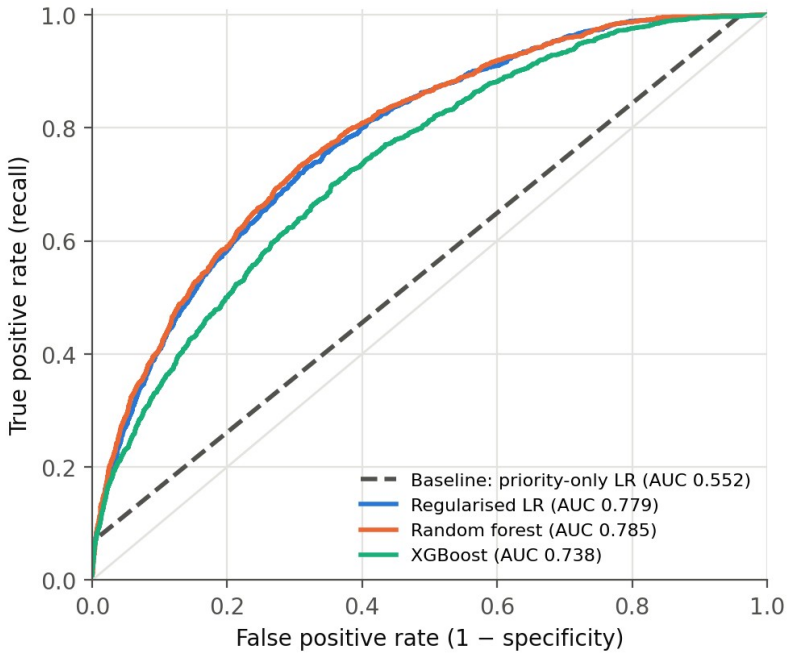
Note. Each fold trains on all earlier training incidents and validates on the next 2,907 incidents; hyperparameters are fixed at the values selected in Table 9. SD = standard deviation.

Visual Evidence

Figure 8 shows the ROC curves. The priority rule hugs the diagonal, because 95% of incidents share the same priority at logging, while the three ML models dominate it at every false-positive rate.

Figure 8

ROC Curves of the Four Models on the Temporal Test Set

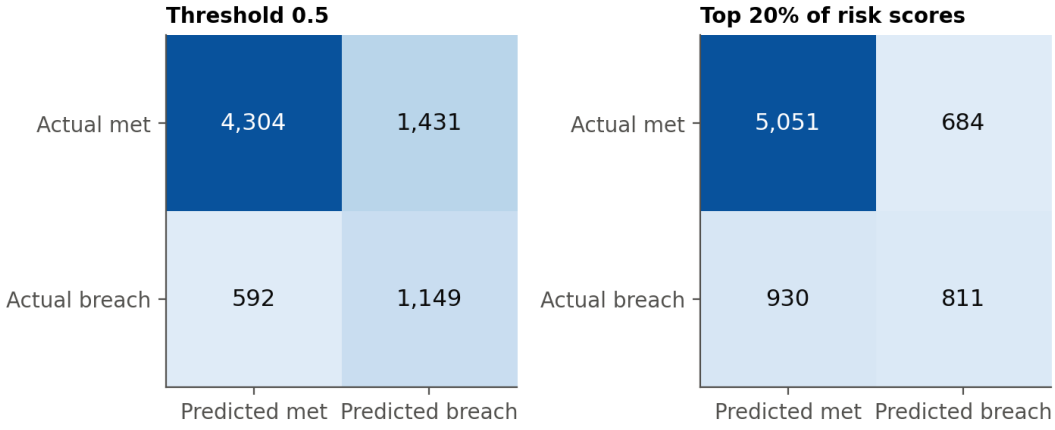


Note. Test set of 7,476 incidents opened after 11:30 on 2 May 2016. The dashed line is the priority-only baseline; the light diagonal is chance (AUC = 0.5).

Figure 9 shows the random forest's confusion matrices. At the default threshold of 0.5 it flags 2,580 tickets and catches 1,149 of 1,741 breaches (recall 0.660, precision 0.445). A service desk is more likely to review a fixed share; flagging the top 20% (1,495 tickets) catches 811 breaches with a precision of 0.542, so more than half the reviewed tickets are real risks, against 23.3% by chance.

Figure 9

Confusion Matrices of the Random Forest at Two Operating Points

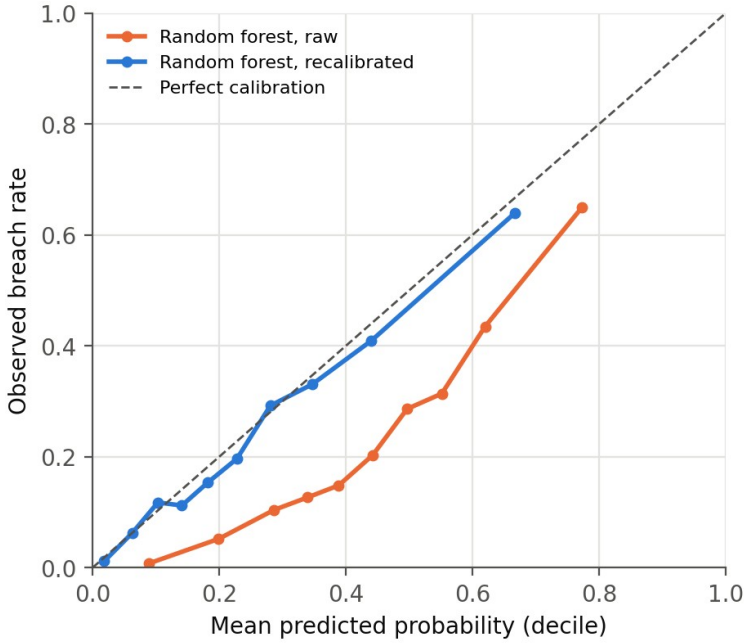


Note. Left: predicted breach when the score is at least 0.5. Right: predicted breach for the 20% highest-risk test incidents. Positive class = breach.

Figure 10 shows calibration. The raw random forest predicts a mean breach probability of 0.419, close to the training rate, while only 23.3% of test incidents breached, so its probabilities are too high throughout. After Platt recalibration on the latest two weeks of training data, the curve lies close to the diagonal, the mean prediction is 0.247, and the Brier score falls from 0.181 to 0.146, below the 0.179 of a constant forecast (Niculescu-Mizil & Caruana, 2005). Calibration matters because the dashboard reports expected breach counts.

Figure 10

Calibration of the Random Forest Before and After Recalibration

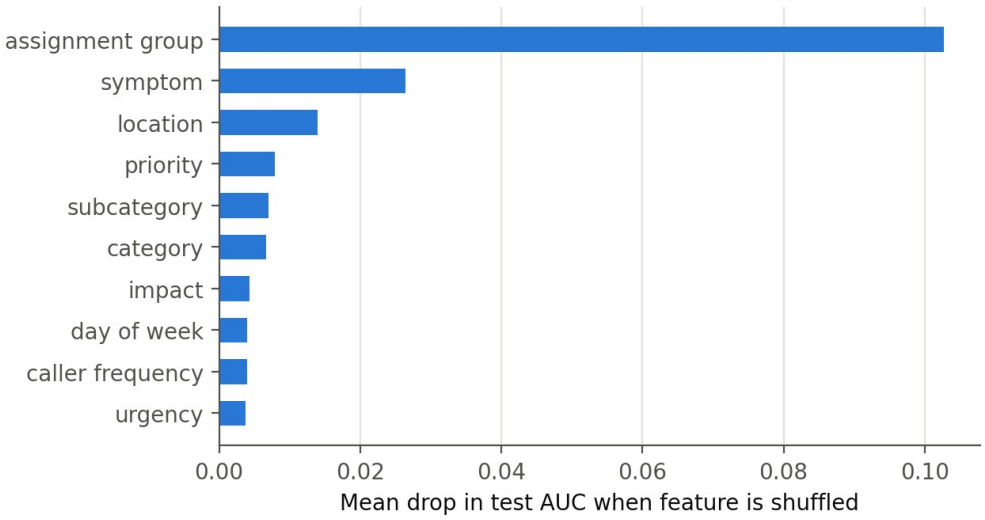


Note. Each point is one decile of predicted probability on the test set. Recalibration uses a logistic (Platt) map fitted on incidents opened 18 April to 2 May 2016.

Figure 11 shows permutation importance. The first assignment group dominates: shuffling it lowers test AUC by 0.103, against 0.026 for symptom, 0.014 for location, and 0.008 for priority. Where a ticket first lands matters far more than its stated urgency.

Figure 11

Top 10 Features of the Random Forest by Permutation Importance



Note. Importance is the mean drop in test-set AUC over five random shuffles of each feature.

Results by Research Question

Table 11 summarises the hypothesis tests and decisions at α = .05.

Table 11

Hypothesis Test Results and Decisions (α = .05)

RQ	Test	Result	Effect size	Decision
RQ1	χ^2 test of independence, priority × breach	$\chi^2(3, N = 24{,}918) = 1{,}293.1$, $p < .001$	$V = 0.228$	Reject H_{01}
RQ2	χ^2 test, reassigned × breach	$\chi^2(1, N = 24{,}918) = 2{,}868.4$, $p < .001$; OR = 4.33 [4.10, 4.58]	φ = 0.339	
	Logistic regression: reassignment count + priority + category	OR = 1.95 [1.90, 2.00] per reassignment, $z = 48.46$, $p < .001$	McFadden $R^2 = 0.217$	Reject H_{02}
RQ3	χ^2 test, knowledge × breach	$\chi^2(1, N = 24{,}918) = 2{,}399.9$, $p < .001$; OR = 6.24 [5.76, 6.76]	φ = 0.310	
	Logistic regression: knowledge + priority + category	OR = 6.62 [6.05, 7.25], $z = 41.01$, $p < .001$	McFadden $R^2 = 0.171$	Reject H_{03}
	Mantel–Haenszel across the two periods (robustness)	OR = 1.17 [1.05, 1.31], $p = .006$		Effect mostly period
RQ4	DeLong test, random forest vs priority LR (one-sided)	ΔAUC = 0.233, $z = 38.45$, $p < .001$	AUC 0.785 vs 0.552	Reject H_{04}

Note. OR = odds ratio with 95% confidence interval in brackets; φ = phi coefficient; V = Cramér's V. Category controls use 31 groups.

RQ1: priority. The breach rate depends strongly on priority, $\chi^2(3, N = 24{,}918) = 1{,}293.1$, $p < .001$, $V = 0.228$, a small-to-medium effect. Five of six pairwise comparisons remain significant after Holm correction; only Critical and High do not differ (98.1% against 99.5%, adjusted $p = .086$). H_{01} is rejected. The association runs opposite to the purpose of priority: the most urgent tickets are the least likely to meet their targets, which suggests that their targets are tighter than the process can deliver.

RQ2: reassignment. After adjustment for priority and category, each additional reassignment multiplies the odds of breach by 1.95 (95% CI 1.90 to 2.00, $z = 48.46$, $p < .001$), and the variance inflation factor of 1.05 shows that this is not explained by the controls. H_{02} is rejected. The effect is not linear (Table 12): compared with no reassignment, one, two, and three or more reassignments raise the odds by factors of 2.88, 5.89, and 12.78, so the first hand-off is the largest single step. The effect differs by priority (likelihood-ratio $\chi^2(3) = 15.0$, $p = .002$): it is strongest for Moderate incidents and about half as strong for Low ones, while Critical and High incidents breach 98% to 100% of the time at every level. Adding the opening week leaves the estimate almost unchanged (OR 1.93), so it is robust to the March break.

RQ3: knowledge-base use. In the pre-specified model, knowledge use is associated with much higher odds of breach (OR 6.62, $p < .001$), so H_{03} is formally rejected. The robustness checks show that this is mostly a period effect (Table 12). Estimated separately, the adjusted odds ratio is 1.15 (95% CI 1.01 to 1.31, $p = .035$) before 21 March 2016 and 1.17 (0.74 to 1.86, $p = .51$) afterwards, and the Mantel–Haenszel estimate across periods is 1.17. Controlling for the week instead reverses the sign (OR 0.61). Once the process change is accounted for, the knowledge flag has at most a small association with breach, and its direction depends on how

time is controlled. The defensible conclusion is that this log does not support a material effect of knowledge-base use, in either direction, on SLA breach.

Table 12

Robustness Checks for RQ2 and RQ3

Check	Estimate [95% CI]	Interpretation
RQ2: banded reassignment (vs 0): 1 / 2 / 3+	OR 2.88 / 5.89 / 12.78	First hand-off is the largest step
RQ2: bands within Moderate priority	OR 2.92 / 5.95 / 13.17	Strongest where volume is
RQ2: bands within Low priority	OR 1.51 / 3.26 / 4.26	Weaker; interaction $p = .002$
RQ2: adjusted for opening week	OR 1.93 [1.88, 1.99] per reassignment	Robust to the March break
RQ3: before 21 March 2016 (n = 5,906)	OR 1.15 [1.01, 1.31], $p = .035$	Small positive
RQ3: from 21 March 2016 (n = 19,012)	OR 1.17 [0.74, 1.86], $p = .51$	Not significant; 0.5% use
RQ3: adjusted for opening week	OR 0.61 [0.44, 0.86], $p = .004$	Sign depends on time control

Note. All models adjust for priority and category. The interaction test is restricted to Moderate and Low incidents (n = 24,240) because Critical and High incidents breach almost without exception, which makes interaction terms inestimable.

RQ4: prediction at logging. The random forest, using only the 17 at-logging attributes, reaches a test AUC of 0.785, significantly above the priority-only baseline's 0.552 ($z = 38.45$, one-sided $p < .001$), so H_{04} is rejected; regularised LR and XGBoost also beat the baseline significantly. Two sensitivity checks support the result. Adding the knowledge flag changes the AUC only to 0.788, and refitting on the stable period alone (from 21 March 2016) gives 0.803 for the random forest against 0.551 for the baseline, so the advantage does not depend on the break. The key relationship identified is the first assignment group, which carries most of the predictive signal (Figure 11).

Overall Interpretation

The results answer the four questions consistently. Priority predicts breach only at the extremes; within Moderate priority, which holds 91% of breaches, it carries no information. Reassignment is the strongest operational driver, and the predictive model confirms its root: the

group a ticket is first sent to determines much of its risk. The knowledge flag, which looked like the second-strongest driver, turned out to reflect a change in recording practice. Model performance is limited mainly by the shift between periods: models trained when 42% of tickets breached were tested when 23% did. This lowered XGBoost's test AUC and inflated raw probabilities, and it was corrected by time-ordered validation and recalibration. The modest gap between regularised LR and random forest also suggests that most of the signal is additive.

Practical Significance

With this sample size significance is guaranteed, so the business test is what the model changes. Reviewing the 20% highest-risk tickets, the random forest finds 46.6% of breaches, and 54.2% of reviewed tickets are real risks; the priority rule at the same workload finds 25.9% at 30.2%. At about 240 incidents a day in May 2016, the list would be about 48 tickets, of which about 26 would otherwise breach. RQ2 turns routing quality into a target: avoiding the first reassignment cuts a Moderate ticket's odds of breach by about two-thirds (OR 2.92). RQ1 gives managers evidence to renegotiate Critical and High targets, which are missed almost every time.

Implementation and User Benefit

The best model was packaged as a working prototype (notebook 06, src/, and app/) that runs on a laptop and can be moved to a cloud service without code changes. It scores tickets in under a second each and is shown here replaying May 2016 as if the tickets were new.

Deployment Approach

The deployable model, rf-v1.0, is the Table 9 random forest fitted on the first 80% of the training period with Platt recalibration on the last 20%, saved with joblib as one 25 MB file that also holds the caller-frequency table and band thresholds. Tickets are High risk when their score is in the top 20% of the recalibration window, Medium for the next 20%, and Low otherwise. On

the 7,476 test incidents it scores an AUC of 0.776, its mean predicted probability (0.247) matches the observed rate (0.233), and the High band holds 22.2% of tickets, with a 50.4% breach rate and 48.1% of all breaches, against 10.2% in the Low band. A batch scorer (src/score_batch.py) scores every ticket logged in a time window into the risk_scores table, and a REST API (app/api.py, FastAPI) scores tickets posted as JavaScript Object Notation (JSON). In production the API would run as a container on a managed cloud service such as Azure App Service, with the model in cloud storage and monthly retraining.

System Integration

Integration requires no change to how tickets are logged. In ServiceNow, an asynchronous business rule on incident insert would send the at-logging fields to POST /score through an outbound REST message and write the returned probability and band to two custom fields, which can then drive assignment rules, a dashboard filter, or a notification to the duty manager. Because the API returns the model version with every score and the risk_scores table records it, every decision can be audited. Where the API cannot be called in real time, the batch scorer can run every few minutes against a reporting replica, as the SQLite prototype does. Retraining reuses src/train_model.py on the latest months of closed incidents, and calibration should be checked monthly because the March 2016 break shows how quickly a process change can move the base rate.

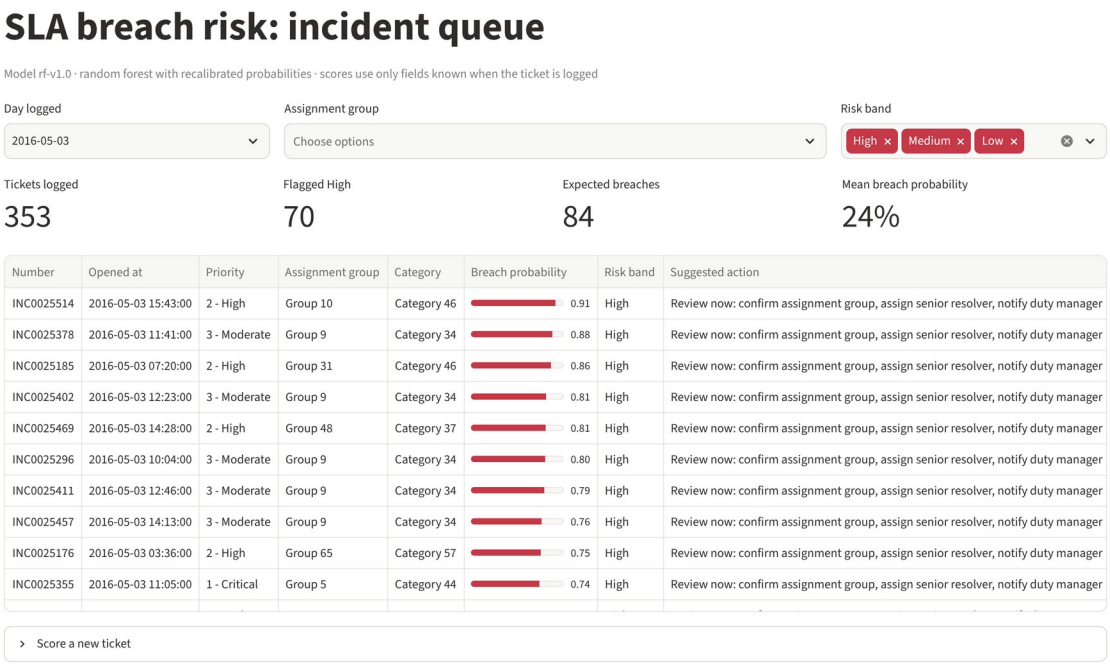
User Interaction

Users interact through the Streamlit dashboard (Figure 12). A duty manager picks a day, can filter by assignment group or risk band, and sees four indicators (tickets logged, tickets flagged High, expected breaches, and mean breach probability) above a queue sorted by risk, with each ticket’s probability, band, and suggested action. A form at the bottom scores a new

ticket from its priority, impact, urgency, assignment group, category, and channel. Other systems interact through the API, which takes the same fields as input and returns the probability, band, and action as output.

Figure 12

Service-Desk Risk Dashboard Showing the Queue for 3 May 2016



Note. Screenshot of app/dashboard.py running locally against the SQLite database after the May 2016 replay. Ticket numbers, groups, and categories are the anonymised codes in the source data.

Benefits to Users

Operational efficiency. Reviewers look at about one ticket in five and find almost half of the breaches, about 1.8 times the yield of reviewing the same number of tickets by priority, so senior attention goes where it changes outcomes. **Financial impact.** Every breach avoided reduces exposure to service credits and the effort of explaining misses in service reviews; even a small reduction in a desk that breaches about a quarter of its tickets is material. **Strategic value.** The risk scores, and the RQ1 and RQ2 evidence, give service delivery managers a quantitative

basis for renegotiating unrealistic targets, for fixing the routing of the assignment groups that drive risk, and for demonstrating proactive management to clients.

Example Use Case

On 3 May 2016 the desk logged 353 incidents. At 15:43 a High-priority ticket in Category 46 was routed to Group 10; the API returned a breach probability of 0.91 and a High band, with the action “confirm assignment group, assign senior resolver, notify duty manager”. By the end of the day the dashboard showed 70 High-risk tickets among the 353 and predicted 84 breaches; 76 actually occurred, and 41 of them were among the 70 flagged tickets (a 58.6% hit rate against 21.5% for the day). A duty manager who checked the routing of those 70 tickets at the next queue review would have been looking at more than half of the day’s breaches while they were still preventable, instead of reading about them in the month-end report.

Limitations and Further Improvements

The study is designed to be transparent about what the data can and cannot show.

Limitations

Data. All incidents come from one anonymised organisation and mostly from three months in 2016. Category and group names are coded, SLA targets are not recorded, and the log contains no free text, which rules out text and LLM methods. A process change in March 2016 shifted the breach rate and the meaning of the knowledge flag. **Method.** RQ1 to RQ3 are observational, so the odds ratios are associations, not causal effects; unmeasured factors such as analyst skill or workload may confound them. **Model.** The tree ensembles fit the training data much better than the test data; results vary slightly with library versions (in Google Colab, XGBoost scored 0.744 to 0.751 rather than 0.738, the random forest 0.784 rather than 0.785, and the deployed model 0.777 rather than 0.776, with no change to any conclusion); permutation

importance can understate correlated features such as category and subcategory; and target encoding of high-cardinality fields (Micci-Barreca, 2001) was not yet tested.

Impact of Limitations

The single-organisation data limits generalisability: the coefficients describe this service desk, and only the method should be transferred. The period shift reduced measured accuracy and would recur after any process change, which is why the deployment plan includes monthly recalibration. The RQ3 conclusion is the most affected, because the period and the knowledge flag cannot be fully separated. Reliability is high: the pipeline is scripted, seeded, and version-pinned, and every number in this report can be regenerated from the repository.

Future Improvements

- Collect data from more organisations and more recent years, with SLA targets and timestamps of each hand-off, to test whether the effects generalise and to estimate time-to-breach.
- Add target encoding and interaction features for assignment group and category, and test an explanation method such as SHapley Additive exPlanations (SHAP) values to show users why a ticket is high-risk.
- Update the risk score during the ticket's life with a sequence model, such as an LSTM or transformer, once reassignments and updates occur (Bukhsh et al., 2021; Tax et al., 2017).
- Run a prospective pilot in which only half of the High-risk tickets are reviewed, to measure the causal effect of the intervention on breaches.

Future Scope

The approach can extend beyond SLA breach to other ticket outcomes, such as reopening or escalation, and to other ITSM processes, such as service requests and changes. Where tickets contain free text, an LLM agent following the ReAct pattern (Yao et al., 2023) could act on the High-risk queue by calling the risk API, retrieving similar past incidents and knowledge articles, and drafting a routing recommendation for the engineer, in the way RCACopilot supports root-cause analysis (Chen et al., 2024). The calibrated risk score would decide where that agent spends its effort, combining the efficiency of a statistical model with the language ability of an LLM.

References

Ahmed, T., Ghosh, S., Bansal, C., Zimmermann, T., Zhang, X., & Rajmohan, S. (2023). Recommending root-cause and mitigation steps for cloud incidents using large language models. In *Proceedings of the 45th International Conference on Software Engineering* (pp. 1737–1749). IEEE. https://doi.org/10.1109/ICSE48619.2023.00149

Amaral, C. A. L., Fantinato, M., & Peres, S. M. (2018a). Attribute selection with filter and wrapper: An application on incident management process. In M. Ganzha, L. Maciaszek, & M. Paprzycki (Eds.), *Proceedings of the 2018 Federated Conference on Computer Science and Information Systems* (Annals of Computer Science and Information Systems, Vol. 15, pp. 679–682). IEEE. https://doi.org/10.15439/2018F126

Amaral, C., Fantinato, M., & Peres, S. (2018b). *Incident management process enriched event log* [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C57S4H

Amaral, C. A. L., Fantinato, M., Reijers, H. A., & Peres, S. M. (2019). Enhancing completion time prediction through attribute selection. In E. Ziemba (Ed.), *Information technology for management: Emerging research and applications* (Lecture Notes in Business Information Processing, Vol. 346, pp. 3–23). Springer. https://doi.org/10.1007/978-3-030-15154-6_1

Bergmeir, C., & Benítez, J. M. (2012). On the use of cross-validation for time series predictor evaluation. *Information Sciences, 191,* 192–213. https://doi.org/10.1016/j.ins.2011.12.028

Botezatu, M. M., Bogojeska, J., Giurgiu, I., Voelzer, H., & Wiesmann, D. (2015). Multi-view incident ticket clustering for optimal ticket dispatching. In *Proceedings of the 21st ACM*

SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 1711–1720). ACM. https://doi.org/10.1145/2783258.2788607

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review, 78*(1), 1–3. https://doi.org/10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2

Bukhsh, Z. A., Saeed, A., & Dijkman, R. M. (2021). *ProcessTransformer: Predictive business process monitoring with transformer network* (arXiv:2104.00721). arXiv. https://doi.org/10.48550/arXiv.2104.00721

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. https://doi.org/10.1145/2939672.2939785

Chen, Y., Xie, H., Ma, M., Kang, Y., Gao, X., Shi, L., Cao, Y., Gao, X., Fan, H., Wen, M., Zeng, J., Ghosh, S., Zhang, X., Zhang, C., Lin, Q., Rajmohan, S., Zhang, D., & Xu, T. (2024). Automatic root cause analysis via large language models for cloud incidents. In *Proceedings of the Nineteenth European Conference on Computer Systems* (pp. 674–688). ACM. https://doi.org/10.1145/3627703.3629553

Cohen, J. (2013). *Statistical power analysis for the behavioral sciences* (2nd ed.). Routledge. https://doi.org/10.4324/9780203771587 (Original work published 1988)

Dang, Y., Lin, Q., & Huang, P. (2019). AIOps: Real-world challenges and research innovations. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion*

Proceedings (ICSE-Companion) (pp. 4–5). IEEE. https://doi.org/10.1109/ICSE-Companion.2019.00023

DeLong, E. R., DeLong, D. M., & Clarke-Pearson, D. L. (1988). Comparing the areas under two or more correlated receiver operating characteristic curves: A nonparametric approach. *Biometrics, 44*(3), 837–845. https://doi.org/10.2307/2531595

Evermann, J., Rehse, J.-R., & Fettke, P. (2017). Predicting process behaviour using deep learning. *Decision Support Systems, 100*, 129–140. https://doi.org/10.1016/j.dss.2017.04.003

Faul, F., Erdfelder, E., Lang, A.-G., & Buchner, A. (2007). G*Power 3: A flexible statistical power analysis program for the social, behavioral, and biomedical sciences. *Behavior Research Methods, 39*(2), 175–191. https://doi.org/10.3758/BF03193146

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters, 27*(8), 861–874. https://doi.org/10.1016/j.patrec.2005.10.010

Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, & A. Oh (Eds.), *Advances in Neural Information Processing Systems 35* (pp. 507–520). Curran Associates. https://doi.org/10.52202/068431-0037

Hanley, J. A., & McNeil, B. J. (1982). The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology, 143*(1), 29–36. https://doi.org/10.1148/radiology.143.1.7063747

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics, 6*(2), 65–70. https://www.jstor.org/stable/4615733

Hosmer, D. W., Jr., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression* (3rd ed.). Wiley. https://doi.org/10.1002/9781118548387

Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data, 6*(4), 1–21. https://doi.org/10.1145/2382577.2382579

Leitner, P., Michlmayr, A., Rosenberg, F., & Dustdar, S. (2010). Monitoring, prediction and prevention of SLA violations in composite services. In *2010 IEEE International Conference on Web Services* (pp. 369–376). IEEE. https://doi.org/10.1109/ICWS.2010.21

Mantel, N., & Haenszel, W. (1959). Statistical aspects of the analysis of data from retrospective studies of disease. *Journal of the National Cancer Institute, 22*(4), 719–748. https://doi.org/10.1093/jnci/22.4.719

Marrone, M., & Kolbe, L. M. (2011). Impact of IT service management frameworks on the IT organization: An empirical study on benefits, challenges, and processes. *Business & Information Systems Engineering, 3*(1), 5–18. https://doi.org/10.1007/s12599-010-0141-5

Micci-Barreca, D. (2001). A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter, 3*(1), 27–32. https://doi.org/10.1145/507533.507538

Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning* (pp. 625–632). ACM. https://doi.org/10.1145/1102351.1102430

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE, 10*(3), Article e0118432. https://doi.org/10.1371/journal.pone.0118432

Shao, Q., Chen, Y., Tao, S., Yan, X., & Anerousis, N. (2008). Efficient ticket routing by resolution sequence mining. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 605–613). ACM. https://doi.org/10.1145/1401890.1401964

Shwartz-Ziv, R., & Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion, 81*, 84–90. https://doi.org/10.1016/j.inffus.2021.11.011

Tax, N., Verenich, I., La Rosa, M., & Dumas, M. (2017). Predictive business process monitoring with LSTM neural networks. In E. Dubois & K. Pohl (Eds.), *Advanced information systems engineering* (Lecture Notes in Computer Science, Vol. 10253, pp. 477–492). Springer. https://doi.org/10.1007/978-3-319-59536-8_30

Teinemaa, I., Dumas, M., La Rosa, M., & Maggi, F. M. (2019). Outcome-oriented predictive process monitoring: Review and benchmark. *ACM Transactions on Knowledge Discovery from Data*, *13*(2), 1–57. https://doi.org/10.1145/3301300

van der Aalst, W. (2016). *Process mining: Data science in action* (2nd ed.). Springer. https://doi.org/10.1007/978-3-662-49851-4

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*. https://arxiv.org/abs/2210.03629

Appendix A: Sample Size Code

The code below, saved as src/sample_size.py in the repository, reproduces Table 4.

```
# QM640 capstone - minimum sample sizes (alpha = 0.05, power = 0.80)
# Dependent variable for all RQs: SLA breach (binary)
import numpy as np
from scipy.stats import chi2, ncx2

def chi2_n(w, df, a=0.05, p=0.80):            # chi-square test of independence
    n, c = 10, chi2.ppf(1 - a, df)
    while ncx2.sf(c, df, n * w**2) < p:
        n += 1
    return n

def auc_test_n(auc, e, prev):                 # RQ4 (Hanley & McNeil, 1982)
    q1, q2 = auc / (2 - auc), 2 * auc**2 / (1 + auc)
    n = 50
    while True:
        pos = round(n * prev); neg = n - pos
        var = (auc*(1-auc) + (pos-1)*(q1-auc**2) + (neg-1)*(q2-auc**2)) / (pos*neg)
        if 1.96 * np.sqrt(var) <= e:
            return n
        n += 1

rq1 = chi2_n(0.10, 3)                         # priority (4) x breach (2): 1091
rq2 = int(np.ceil(chi2_n(0.10, 1) / (1 - 0.047)))  # 785 / 0.953 -> 824
rq3 = int(np.ceil(chi2_n(0.10, 1) / (1 - 0.154)))  # 785 / 0.846 -> 928
test = auc_test_n(0.75, 0.03, 9115 / 24918)   # observed prevalence 0.366: 1191
rq4 = int(np.ceil(test / 0.30))               # 3970
print(rq1, rq2, rq3, test, rq4, "final N =", max(rq1, rq2, rq3, rq4))
```

Appendix B: Notebook Map and Run Order

Run the notebooks in order after installing requirements.txt; each writes its results to results/ and its figures to figures/.

Table B1

Notebooks, Purpose, and Outputs

Notebook	Purpose	Produces
01_data_preparation.ipynb	Load, validate, collapse to incidents, derive variables	data/processed/incidents.csv
02_cleaning_and_eda.ipynb	Cleaning checks, summary statistics, EDA figures	Figures 3–6; results/eda.json
03_rq1_rq3_hypothesis_tests.ipynb	χ^2 tests, logistic regression, Mantel–Haenszel, week checks	Table 11 (RQ1–RQ3); results/rq1_rq3.json
04_rq4_prediction_models.ipynb	Tuning, test evaluation, DeLong test, importance	Table 9; Figures 8, 11; results/rq4.json
05_extended_evaluation.ipynb	Train vs test, forward-chaining CV, calibration, top-k%, RQ2/RQ3 robustness	Tables 6, 10, 12; Figures 9, 10; results/final_eval.json
06_deployment_prototype.ipynb	Database, saved model, batch scoring, API test	Table 5; results/deployment.json
src/sample_size.py	Minimum sample size	Table 4

Appendix C: Running the Deployment Prototype

From the repository root, after installing requirements.txt:

```
python src/build_database.py          # builds data/db/itsm.sqlite
python src/train_model.py              # saves models/sla_breach_rf.joblib
python src/score_batch.py --start 2016-05-02 --end 2016-06-01
uvicorn app.api:app --port 8000        # REST API; docs at http://localhost:8000/docs
streamlit run app/dashboard.py         # dashboard at http://localhost:8501

Example request body for POST /score:
[{"number": "INC_DEMO", "opened_at": "2016-05-03 09:15:00",
  "log_priority": "3 - Moderate", "log_impact": "2 - Medium", "log_urgency": "2 -
Medium",
  "log_assignment_group": "Group 70", "log_category": "Category 26",
"log_contact_type": "Phone"}]

Response:
[{"number": "INC_DEMO", "breach_probability": 0.303, "risk_score_raw": 0.528,
  "risk_band": "Medium", "suggested_action": "Watch: check at next queue review"}]
```